Architecture modulaire et ROS 2

EIGSI · 5e année

Par Ludovic DELVAL

09/11/2026

IRT JULES VERNE
LE FUTUR DE VOS USINES

Introduction et échanges



Ludovic DELVAL

Ingénieur de recherche à l'IRT Jules Verne (Nantes)

Utilisateur de ROS depuis environ 10 ans

Formateur ROS auprès d'industriels

Et vous ? Qu'attendez-vous de cette intervention ?

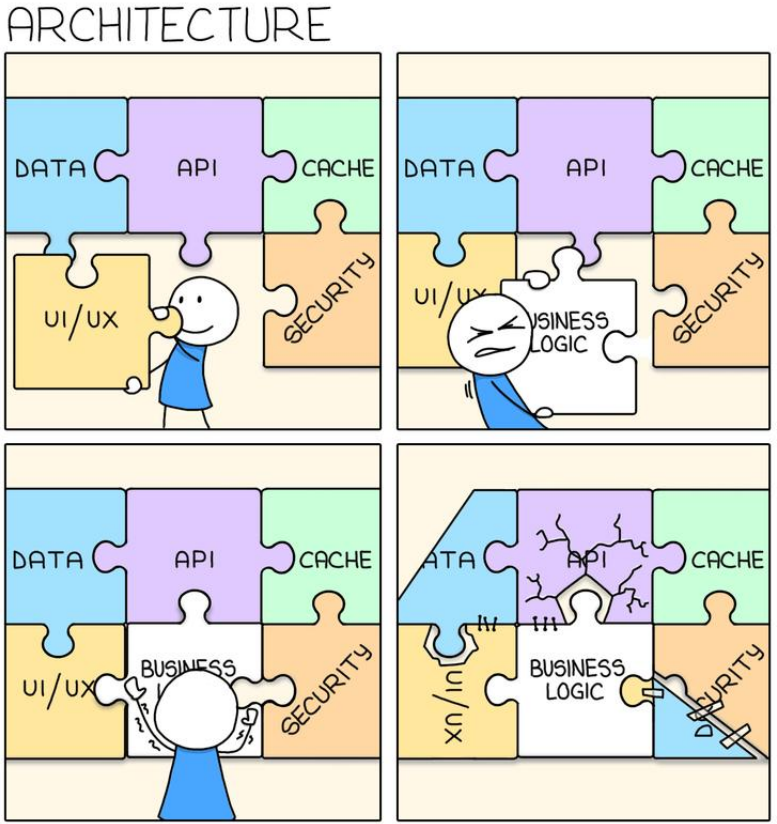
Objectifs

1. Comprendre pourquoi une architecture modulaire est importante en robotique et comment la réaliser

2. Avoir les bases de ROS 2 et comprendre le fonctionnement

3. Utiliser ROS 2 pour faire une application simple

Architecture Modulaire ?



Pourquoi modulariser ?



ISS : plusieurs éléments assemblés autour du module Tranquility.

Image : NASA, ISS067-E-214089 (2022) · [V1]

Changer une partie sans remettre en cause tout le système.

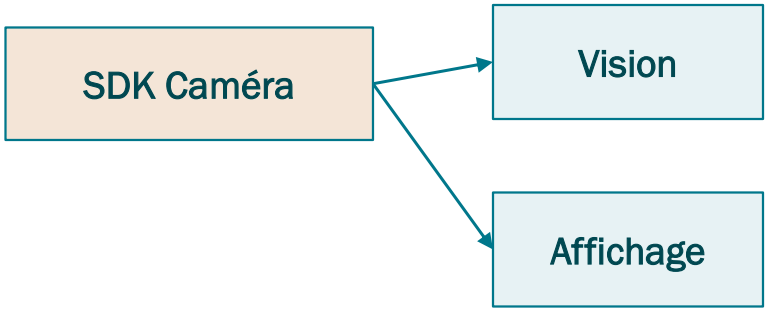
Limiter la propagation des changements.

Tester un module de manière séparée.

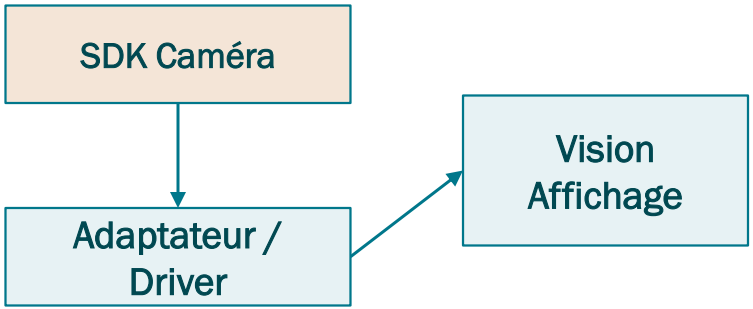
Permettre un travail d'équipe avec des limites claires.

Le coût des dépendances

Dépendances directes



Dépendance isolée



Une modification locale peut avoir un effet global.

Le risque vient aussi des connaissances partagées.

Une frontière utile réduit les hypothèses à coordonner.

1968 : Dijkstra et les niveaux d’abstraction

Logique de l’application 

Services abstraits 

Accès aux ressources 

Chaque niveau utilise
les garanties du niveau inférieur.

Le système organise des responsabilités en niveaux.

Les niveaux supérieurs utilisent des abstractions.

La structure aide à raisonner et à tester progressivement.

Source : [H2] Dijkstra, 1968 · référence complète dans les notes.

Les bénéfices ont un prix

Plus d'abstractions ➔ Plus d'interfaces à définir et maintenir.

Plus de modules ➔ Des intégrations et des diagnostics à organiser.

Plus de formats de données ➔ Des conversions ou communications parfois coûteuses.

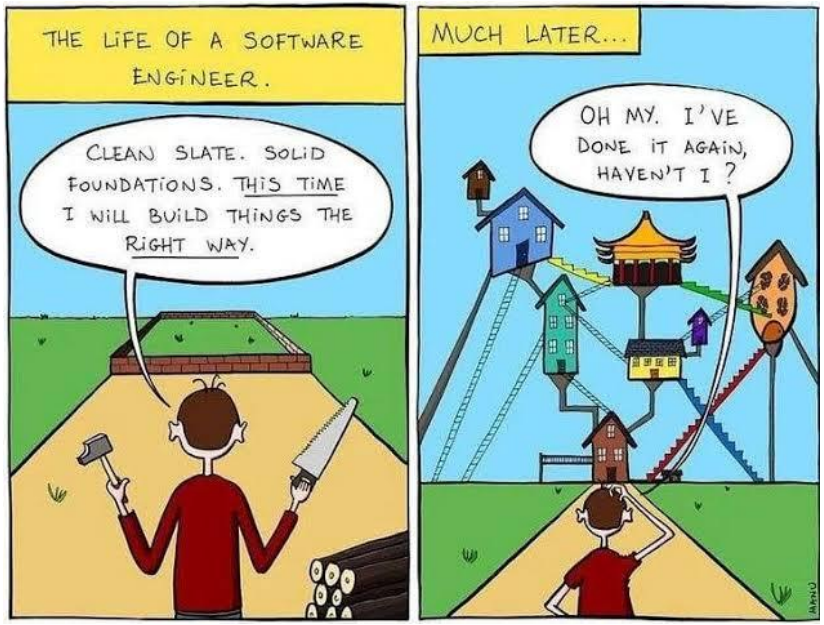
➔ L'architecture est à proportionner aux changements probables.

Fonction, module et architecture

La sémantique :

Fonction : ce qu'un système doit faire.
Module : une responsabilité derrière une interface.
Architecture : un ensemble de modules, leurs relations et les règles de composition.

Comment faire une bonne architecture modulaire ?



Bien définir les responsabilités

L'architecture est un outil de communication – Il faut que les informations soient claires !

Exemples :

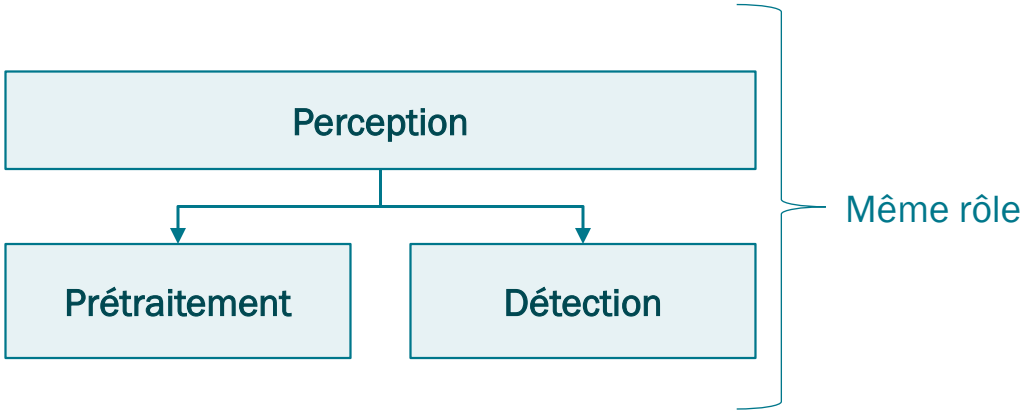
Module d'acquisition : fournir une observation exploitable.

Module de perception : identifier et localiser une pièce.

Module de coordination : choisir la prochaine tâche.

Module d'exécution : réaliser la tâche acceptée.

La cohésion

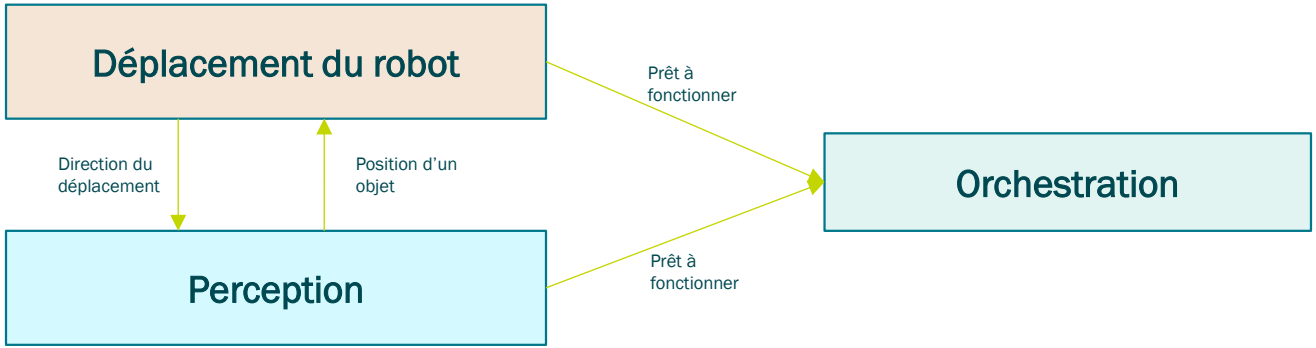


Un module regroupe des éléments au service du même rôle.

Prétraitement et détection peuvent évoluer ensemble.

Déplacer le robot relève d'une autre responsabilité.

Le couplage : maîtriser les dépendances



Les modules doivent coopérer pour réaliser le système.
Le couplage décrit leurs dépendances : données, temps, état ou choix techniques.

Une interface explicite rend ces dépendances visibles et maîtrisables.

Abstraction et encapsulation

Bien définir les frontières des modules

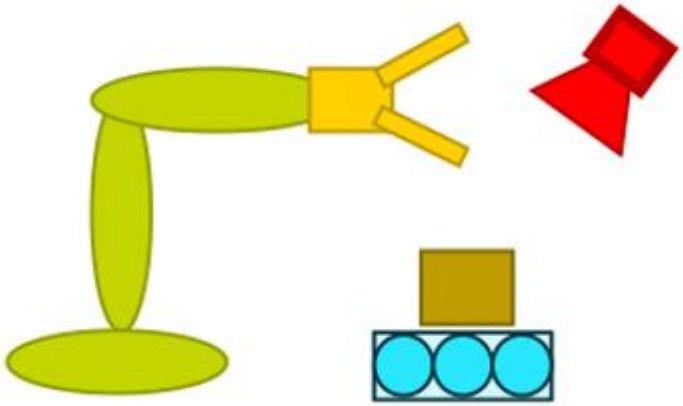


Abstraction : exprimer le service utile à l'utilisateur.

Encapsulation : contrôler l'accès à l'intérieur du module.

Masquage d'information : isoler une décision susceptible de changer.

Le test du changement

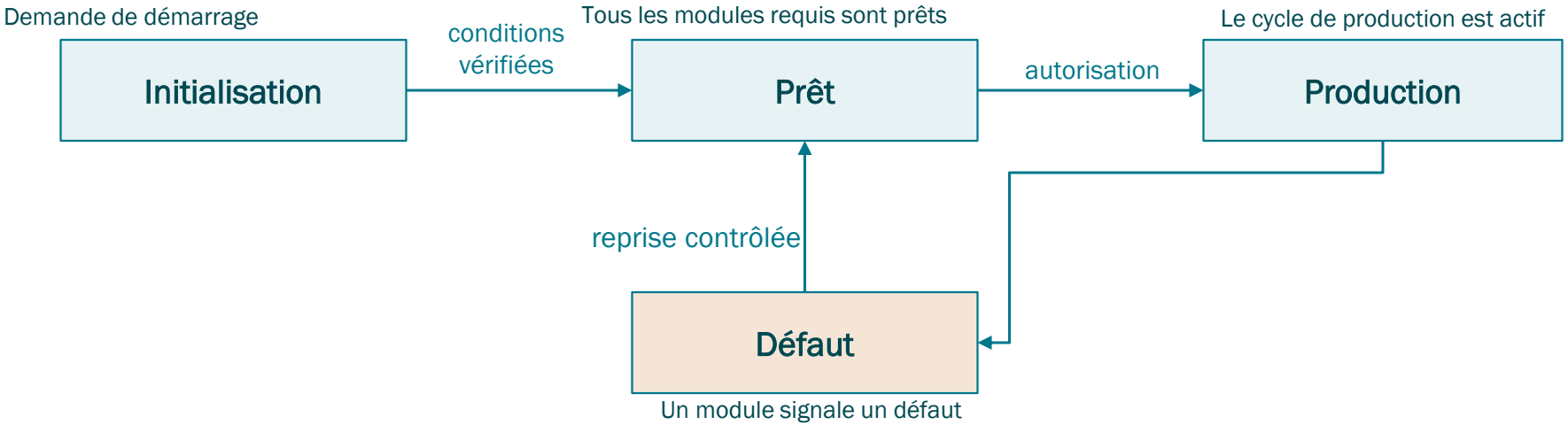


Une nouvelle caméra : quelle connaissance est concernée ?

Une nouvelle recette : quelle règle évolue ?

Un nouveau robot : quelle interface peut rester stable ?

Un responsable pour chaque état



Un module possède l'état global du cycle.

Les composants exposent leurs états locaux.

Les demandes de transition passent par une règle explicite.

Décision, exécution et supervision

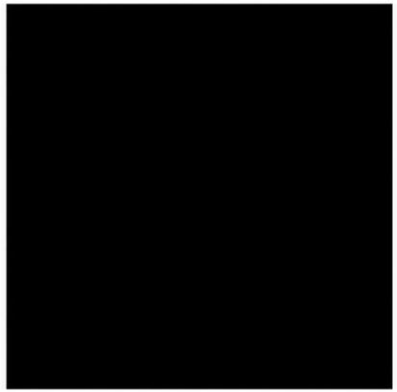


La décision choisit le résultat à atteindre.

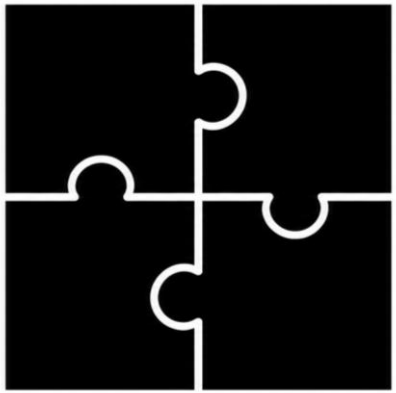
L'exécution transforme la demande en opération.

La supervision rend les états et défauts compréhensibles.

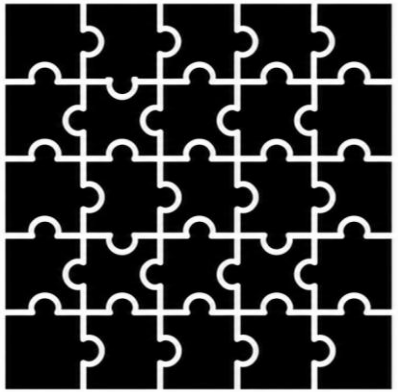
La bonne granularité



Difficile à faire évoluer



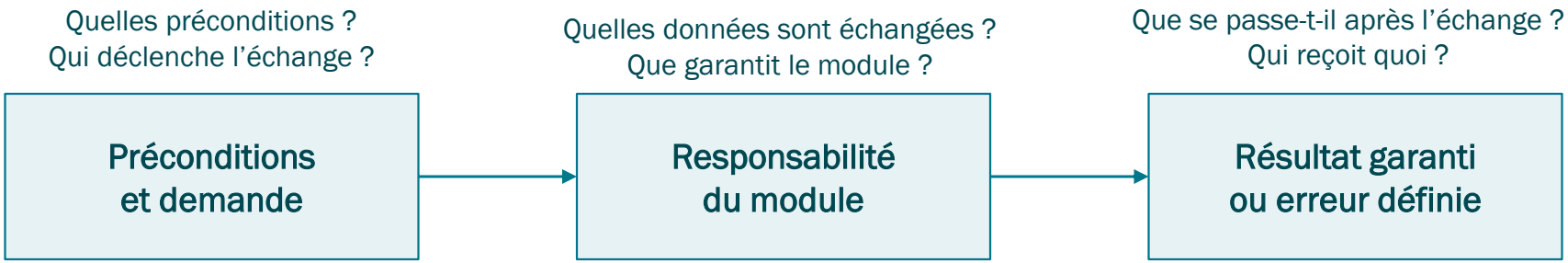
Un compromis à justifier



Difficile à intégrer

Trop large : changements et tests deviennent difficiles à isoler.
Trop fin : intégration et échanges deviennent disproportionnés.
La frontière se justifie par une responsabilité et un besoin.

Le contrat d'interface



Sens, temps et règles de validité accompagnent les échanges.

Données, unités et repères.

Conditions temporelles et validité.

Résultats, erreurs et comportement attendu.

Un exemple : la position d’une pièce

Identifiant et catégorie : quels types ? Quelles valeurs autorisées ?
Position et orientation : quels formats, unités et repère ?

Horodatage et validité : quelle horloge ? Quel âge maximal ?
Statut : valide, pièce absente ou erreur ? Quel niveau de confiance ?

Même format, sens différent



Analogie : l’aspect d’un connecteur ne décrit pas tout le contrat.

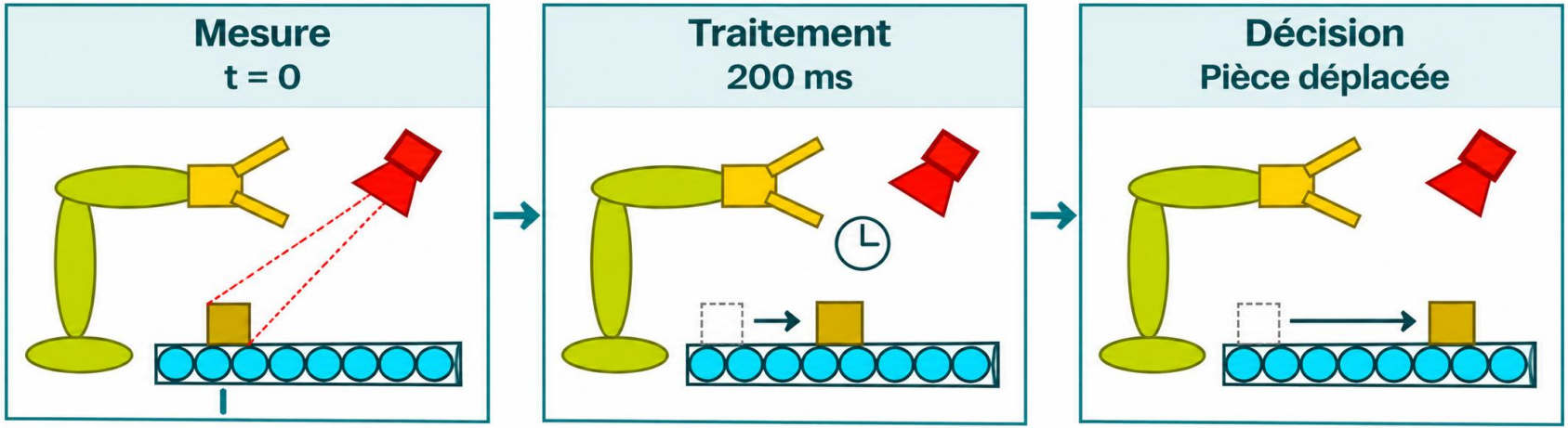
100 millimètres et 100 mètres ont le même type numérique.

Une position caméra n’est pas une position robot.

« Aucune pièce » diffère de « aucune donnée reçue ».

Photo : Cjp24 / Wikimedia Commons · CC BY-SA 3.0 · [V4]

Le temps fait partie du contrat



0,5 m/s × 0,2 s = 0,10 m

Les formes d’échange

Publication / souscription : un flux d’observations pour un ou plusieurs consommateurs.
Requête / réponse : une opération et son résultat associé.

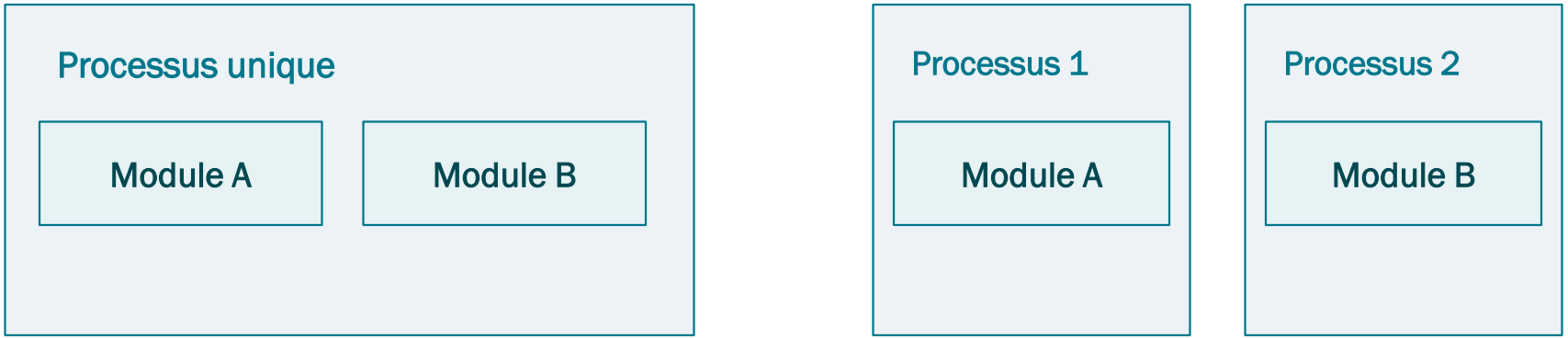
Tâche suivie : acceptation, progression, résultat et demande d’annulation.

Erreurs et évolution du contrat

Que faire si la réponse arrive trop tard ?
Un nouvel essai risque-t-il de répéter une opération ?

Comment annoncer une modification incompatible ?
Quels tests vérifient le comportement attendu ?

Modularité et déploiement



Une architecture logique décrit les responsabilités.

Le déploiement choisit processus et machines.

Plusieurs modules peuvent partager un même processus.

Un module remplaçable

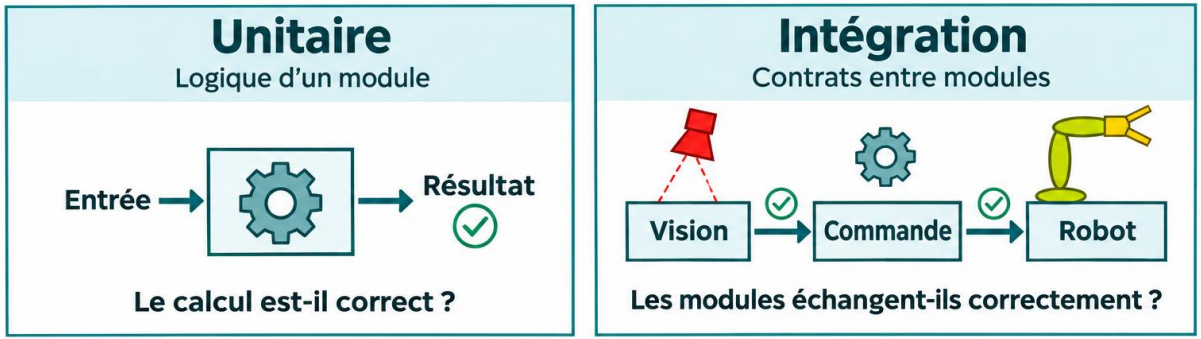


Une interface stable permet plusieurs implémentations.

La substitution doit préserver les garanties utiles.

La simulation et le matériel peuvent partager un contrat.

Tester à plusieurs niveaux

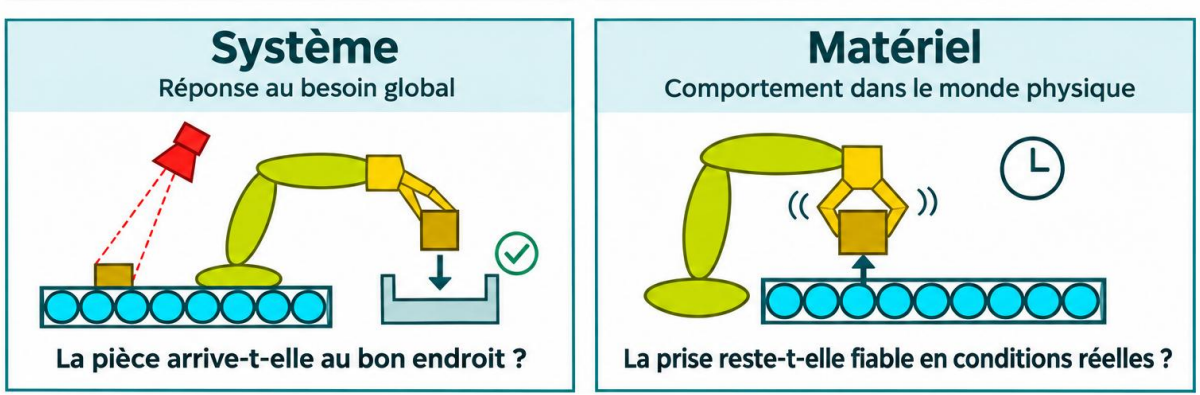


Des questions différentes à chaque niveau

Unitaire : logique d'un module.

Intégration : contrats entre modules.

Tester à plusieurs niveaux



Des questions différentes à chaque niveau

Système : Réponse au besoin global

Matériel : Comportement dans le monde physique.

Un essai dans des conditions représentatives



Essai du bras de Curiosity au JPL, le 3 septembre 2010.

Photo : NASA/JPL-Caltech (2010) · [V5]

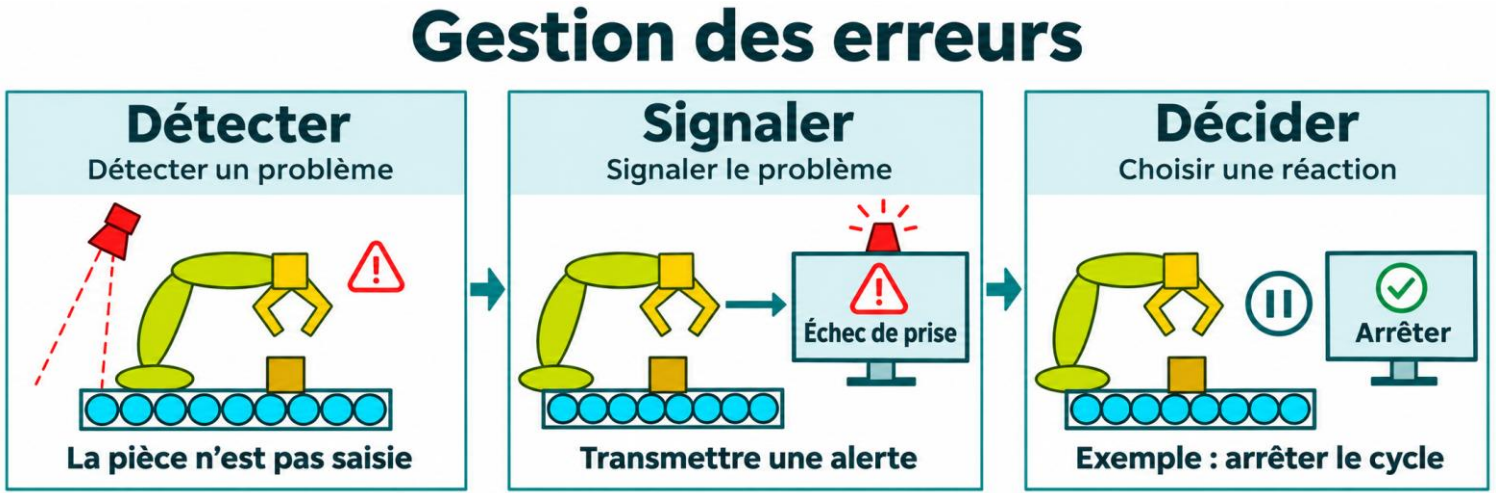
Curiosity : essai du bras sur une table inclinée à 20°.

Le montage met en jeu le bras, le rover et la gravité.

Question : que ne vérifierait pas un test de logique isolé ?

Détecter, signaler et décider

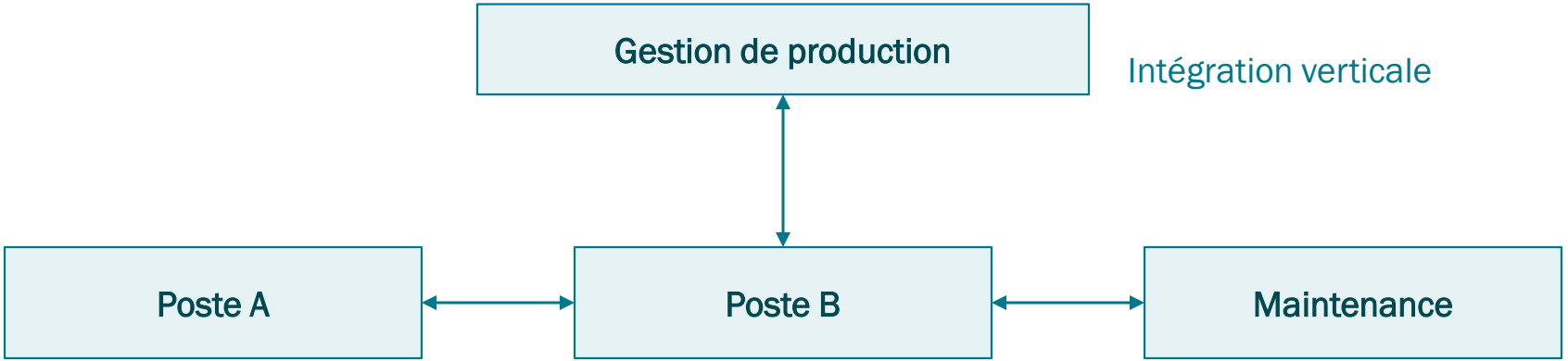
Gestion des erreurs



Méthode de conception d’une architecture modulaire

1. Définir les scénarios et les contraintes. Que doit-il se passer ?
2. Répartir les responsabilités et anticiper les changements. Qui fait quoi ?
3. Définir les contrats et les états. Qui échange quoi, avec quelles garanties ?
4. Prévoir les défauts et les tests. Comment vérifier la réaction attendue ?

Industrie 4.0 : élargir la coopération



Coopération horizontale

Relier les équipements, les applications et les partenaires.

Échanger des informations compréhensibles entre fabricants.

Faire évoluer la production et suivre le cycle de vie.

[I1] IDTA ; [I4] OPC Foundation · schéma pédagogique original

Les idées essentielles

Découper selon les responsabilités et les évolutions attendues.

Garder explicites les dépendances nécessaires.

Définir le sens, le temps et les erreurs aux interfaces.

Prévoir les tests et les défauts avant l'intégration.

De la modularité à ROS 2

Des responsabilités peuvent devenir des nœuds au rôle explicite.
Les contrats se traduisent en interfaces et échanges typés.

Les packages regroupent du code et des ressources réutilisables.
ROS facilite l'intégration. Le bon découpage reste à concevoir.



ROS 2 – les bases



Objectifs

Nœuds, topics, services, actions et interfaces

Packages, workspace, paramètres et lancement

Choisir un échange et lire un graphe de communication

Utiliser la documentation et orienter un diagnostic

ROS : concepts et outils

- Bibliothèques et outils pour la robotique



Couche Middleware

ROS

Couche Logicielle

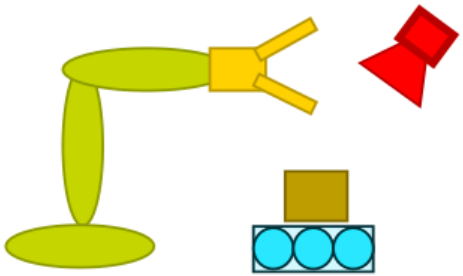


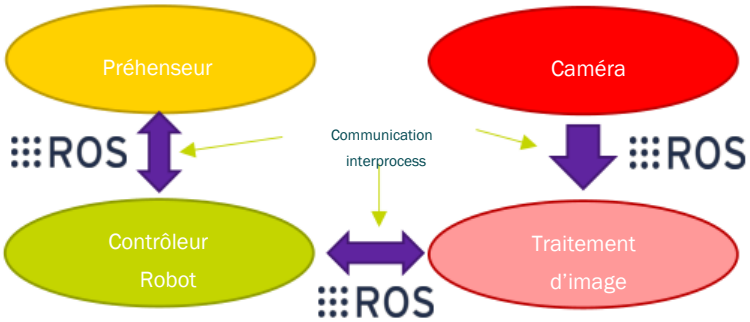


Communication entre processus | Outils | Compétences | Écosystème Communautaire

Communication entre composants

- Cas Pick and Place



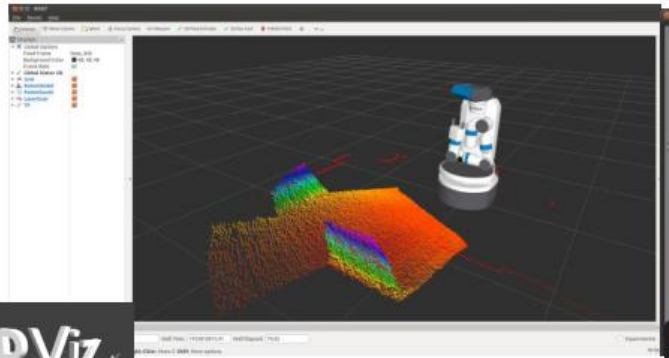


ROS 2 permet aux nœuds d'échanger des données typées.

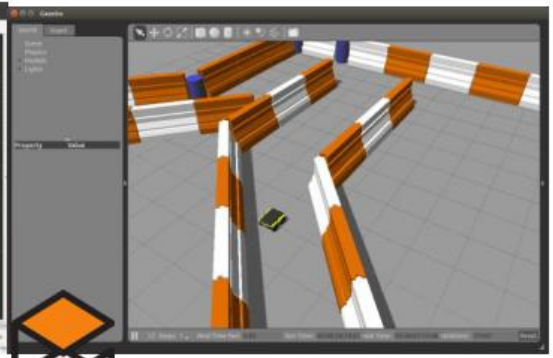
Topics, services et actions répondent à des besoins différents.

Des outils logiciels

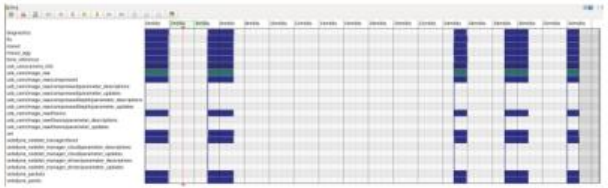
- Accélère le développement



Outils de visualisation

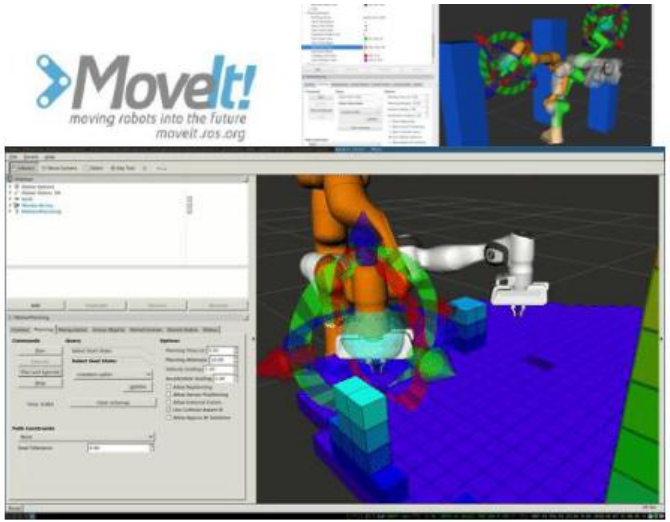


Outils de simulation

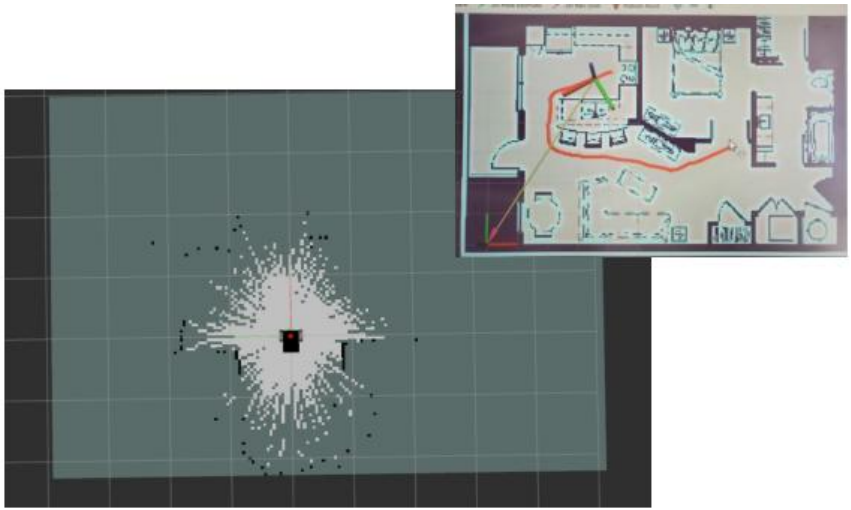


Enregistrement et relecture de données

Algorithmes et composants réutilisables



Planificateur de trajectoires
(avec évitement d'obstacle)

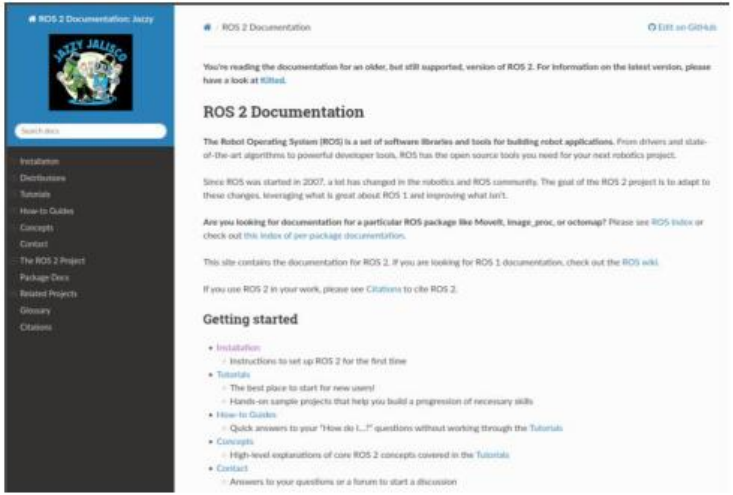


Outils de cartographie et de navigation 2D

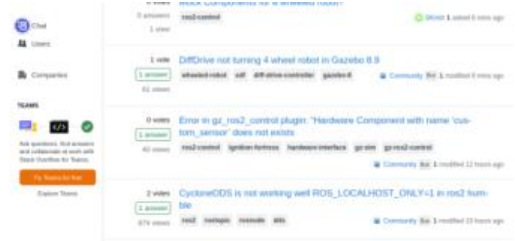
Une grande communauté

ROS.org

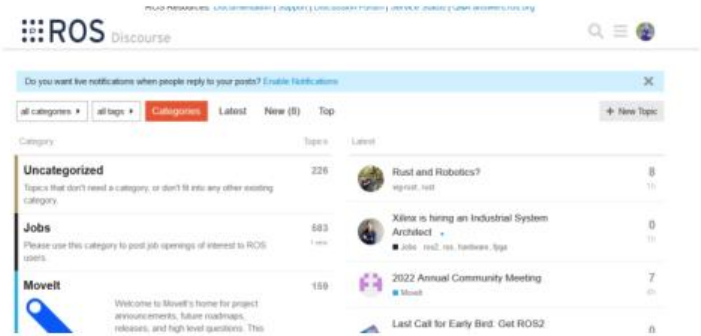
Site principal



https://docs.ros.org/en/jazzy/index.html



Questions-réponses (Robotics Stackexchange)



Forum de la communauté (Discourse)

Pourquoi ROS ?

1. Profiter d'algorithmes open source à l'état de l'art
2. Faciliter l'intégration avec différents matériels
3. Partager les données facilement

« Les robots et les défis robotiques sont tellement complexes qu'ils deviennent impossibles à relever seul »

O.Stasse, directeur de l'équipe GEPETO du CNRS LAAS,

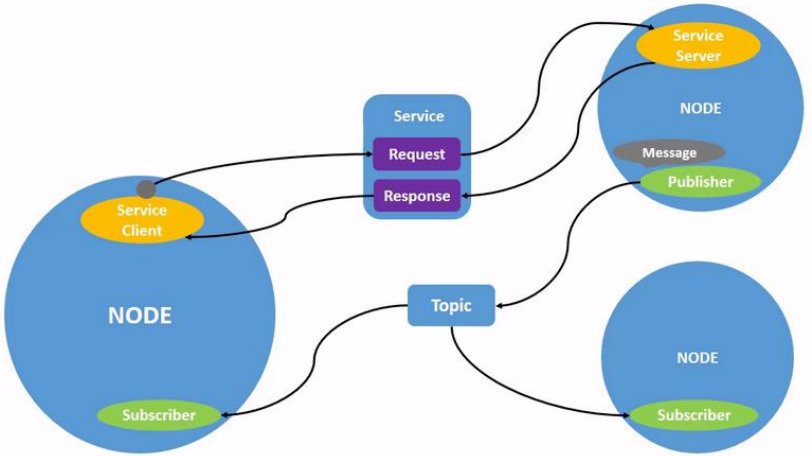
Journée Francophone ROS 2017

Nœud, exécutable, processus

Un nœud porte un rôle et participe au graphe ROS.
Un exécutable est un programme que l'on peut lancer.

Un processus est une instance de programme en cours d'exécution.

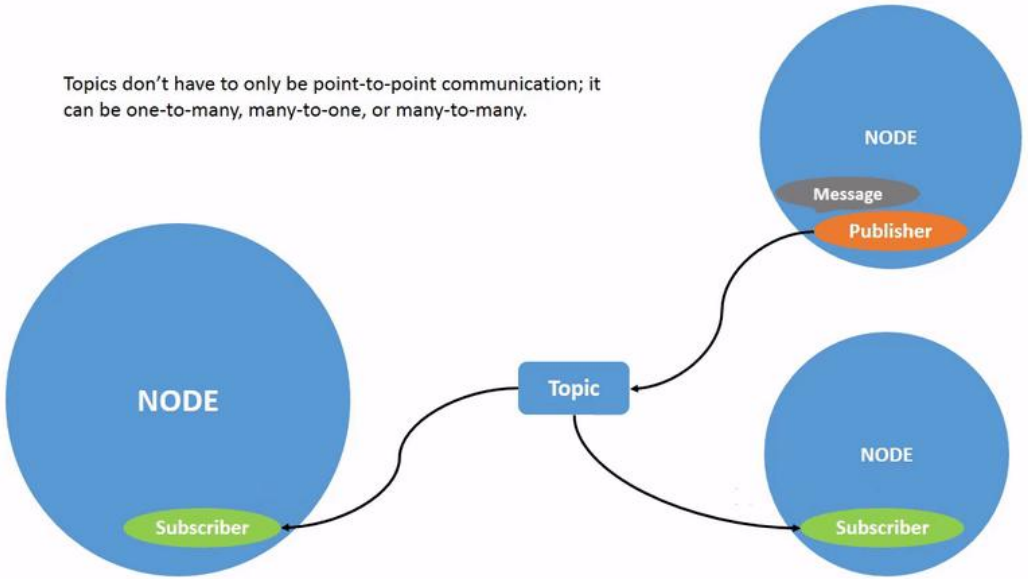
Il peut héberger plusieurs nœuds. Les nœuds peuvent fonctionner sur plusieurs machines.



Topics : diffuser un flux de données

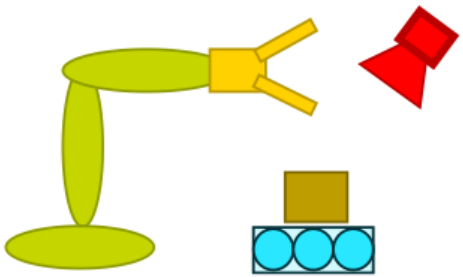
Publication / souscription sur un nom de topic et un type de message.
Un ou plusieurs producteurs et consommateurs.

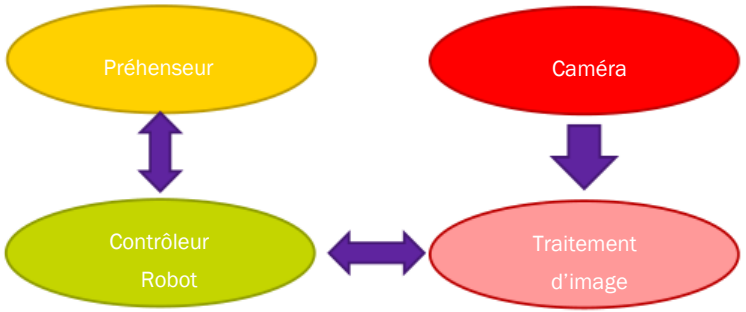
Exemples : image caméra, position, vitesse mesurée.
Même nom résolu, même type et QoS compatibles.



Cellule de manipulation : les échanges

- Pick and place



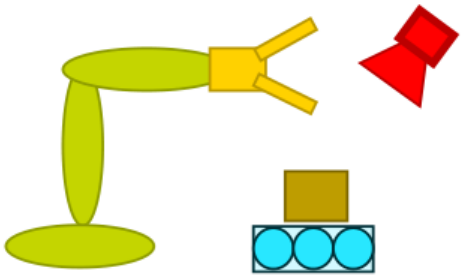


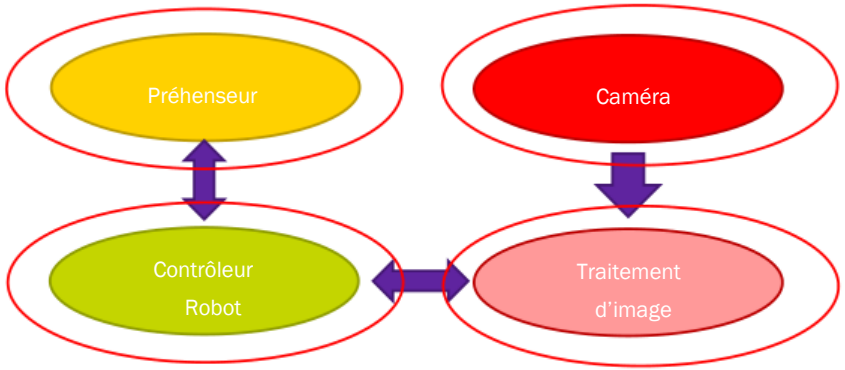
Trouvez les nœuds ?

Trouvez les topics ?

Cellule de manipulation : les nœuds

- Pick and place

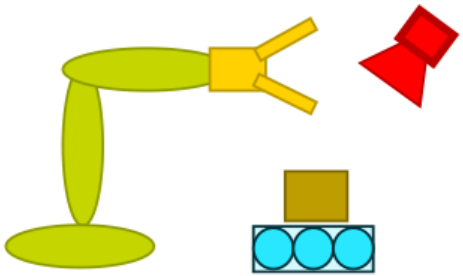


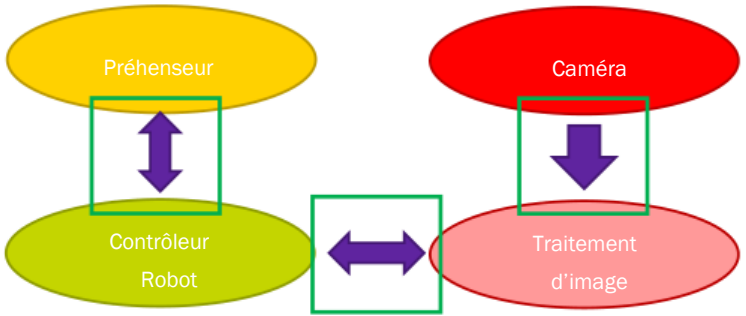


Les nœuds sont les éléments qui communiquent ensemble

Cellule de manipulation : les topics

- Pick and place





Les topics sont les lieux d'échanges des données

Les messages

Une structure de données

```
# This contains the position of a point in free space
float64 x
float64 y
float64 z
```

Compact Message Definition

```
float64 x
float64 y
float64 z
```

```
# A representation of pose in free space, composed of posi
Point position
Quaternion orientation
```

Compact Message Definition

```
geometry_msgs/Point position
geometry_msgs/Quaternion orientation
```

Défini dans un fichier .msg

- Peut contenir plusieurs champs simples.
- Peut être composé d'autres messages.

Il existe des messages « communs » et des messages « standards »

- common_interfaces (ROS 2) => Liés à leur utilisation (Pose, Image, …)
- std_msgs (ROS 2) => Format de données « standards » (Booléen, Tableau de float64, …)

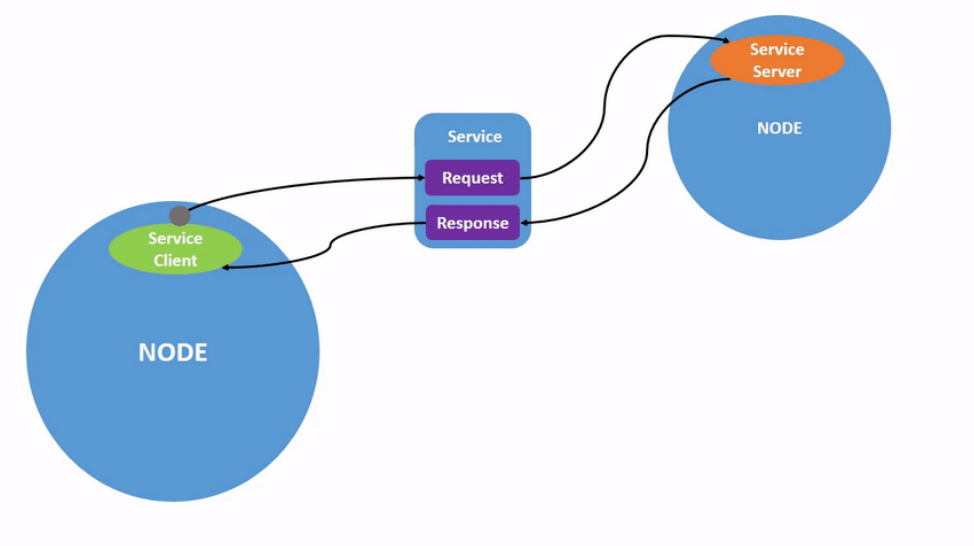
Services : demander un résultat

Une requête, puis une réponse typée

Pour une opération courte : lire un état, réinitialiser

Un client peut appeler un service de façon asynchrone

Éviter de bloquer un callback en attendant une réponse.



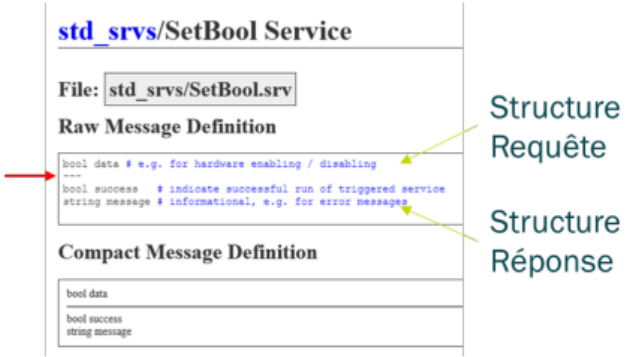
La structure des services (.srv)

Structure de données double

Défini dans un fichier .srv

- Même construction de données qu'un .msg :
  types simples ou autres messages.
- La requête et la réponse sont séparées
  par une ligne « --- ».

Des packages d'interfaces, des algorithmes et des outils ROS fournissent des types de services.



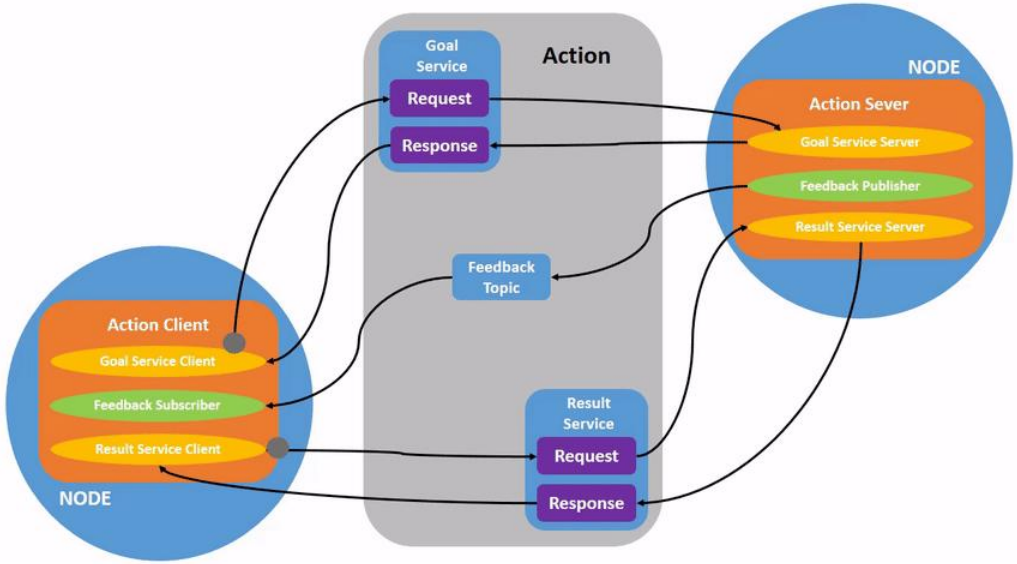
Actions : suivre une tâche longue

Un objectif accepté ou refusé par le serveur

Des retours de progression et un résultat final

Une demande d'annulation peut être envoyée

Exemples : rejoindre une pose, exécuter une trajectoire.



À vous : choisir le bon échange

Recevoir une image caméra en continu : topic, service ou action ?
Réinitialiser un compteur : quel échange ?

Calculer une trajectoire : quelles exigences préciser avant de choisir ?
Exécuter un déplacement avec suivi et annulation : quel échange ?

Recevoir la position du robot en continu : quel échange ?

QoS : choisir le comportement des échanges

Fiabilité : best effort accepte des pertes ; reliable prévoit des retransmissions.
Historique : avec keep last, depth limite le nombre d’échantillons conservés.

Durabilité : transient local peut fournir des données à un abonné tardif.
Les profils doivent être compatibles. Reliable ne garantit pas un délai maximal.

Paramètres : configurer un nœud

Valeurs nommées, propres à chaque nœud

Exemples : seuil de détection, fréquence, couleur

ros2 param list / get / set / describe

Une modification peut être refusée par le nœud.

Noms, espaces de noms, remapping

Un nom complet identifie un élément du graphe

Un namespace regroupe les noms avec un préfixe : /robot1/position, /robot2/position

Un remapping adapte un nom sans modifier le code

Construire et retrouver les fichiers

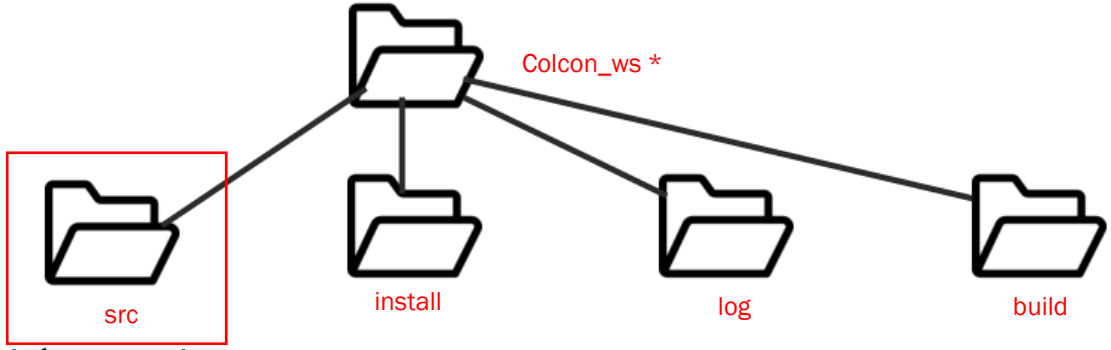
Un package ROS 2 regroupe du code, des ressources et des métadonnées.
colcon orchestre la construction des packages.

ament fournit des outils et conventions de construction.
L'environnement actif détermine les packages visibles.

Les workspaces

Les dossiers d'un workspace

Un workspace est l'endroit où sont stockés et organisés les fichiers ROS



src : sources organisées en packages.
build : fichiers intermédiaires de construction.
install : exécutables, ressources et scripts d'environnement.
log : journaux des commandes de construction.

Package

Une unité d'organisation du logiciel ROS

Un package regroupe du code, des ressources et des métadonnées.

On peut l'obtenir de deux manières :

- Installer un paquet préconstruit avec le gestionnaire de l'environnement (par exemple apt sous Ubuntu, ou Pixi pour le kit du cours).
- Construire les sources dans un workspace, puis utiliser leur installation. Le code source reste dans le dossier src.

Pour un package ROS 2, package.xml décrit notamment les dépendances et le type de construction.
Les fichiers de construction dépendent du langage et du type de package.

Le manifeste package.xml

Le fichier package.xml

package.xml décrit le package, ses dépendances et son type de construction.
C'est sa carte d'identité.

Déclare les dépendances nécessaires au package.
À compléter par un environnement maîtrisé.

xml — Copier le code

```xml
<package format="3">
  <name>my_package</name>
  <version>0.0.0</version>
  <description>My ROS 2 package</description>
  <maintainer email="your_email@example.com">Your Name</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <buildtool_depend>ament_python</buildtool_depend>

  <exec_depend>rclpy</exec_depend>
  <exec_depend>std_msgs</exec_depend>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>
</package>
```

Windows + Pixi : environnement du cours

Kit préparé : pixi.toml et pixi.lock

Depuis la racine du kit : pixi run ros2 --help

Dans chaque terminal : même kit et même domaine

Après compilation : charger l’overlay du workspace.

Démonstration : du source au nœud

Montrer package.xml et le point d’entrée

Construire, charger l’overlay, lancer le programme

Modifier une sortie puis reconstruire et relancer.

La ligne de commande ROS 2

Le B-A-BA de ROS

Les lignes de commande permettent de faire énormément de choses sur ROS :

- Lancer un ou plusieurs exécutables
- Enregistrer ou relire des données échangées sur le réseau
- Faire de l'introspection sur ce qu'il se passe actuellement
- Publier / Ecouter sur des topics
- Appeler des services
- ...

Toutes les commandes ROS commencent par :

```
ros2
```

Exemple

```
ros2 --help
```

Lancer un programme

Lancer un programme

- **Exécute un programme du package (doit être exécutable)**

```
ros2 run      package_name nom_du_programme                arguments
```

- **Lance le fichier launch (et les programmes associés)**
  - Possibilité de passer les arguments déclarés (utiliser « :=»)

```
ros2 launch      package_name fichier_launch
```

Inspecter et publier des topics

Les topics

```
ros2 topic info        nom_du_topic -v
```

- Donne toutes les informations sur le topic (publishers, subscribers, type, QoS)

```
ros2 topic echo        nom_du_topic
```

- Souscrit au topic et affiche le contenu des messages reçus

```
ros2 topic pub        nom_du_topic type_de_message "{contenu}"                    -r    X
```

- Crée un publisher qui publie le contenu à une fréquence de X hertz.
  - Il faut mettre le contenu sous forme de syntaxe YAML.

Lister les topics et mesurer leur fréquence

Les topics

```
ros2 topic list
```

- Liste les topics du domaine ROS actif.

```
ros2 topic hz       nom_du_topic
```

- Mesure la fréquence reçue par cet outil, qui dépend aussi de ses QoS.

Inspecter les nœuds

Les nœuds

```
ros2 node info        nom_du_nœud
```

- Décrit les interfaces du nœud : topics, services et actions.

```
ros2 node list
```

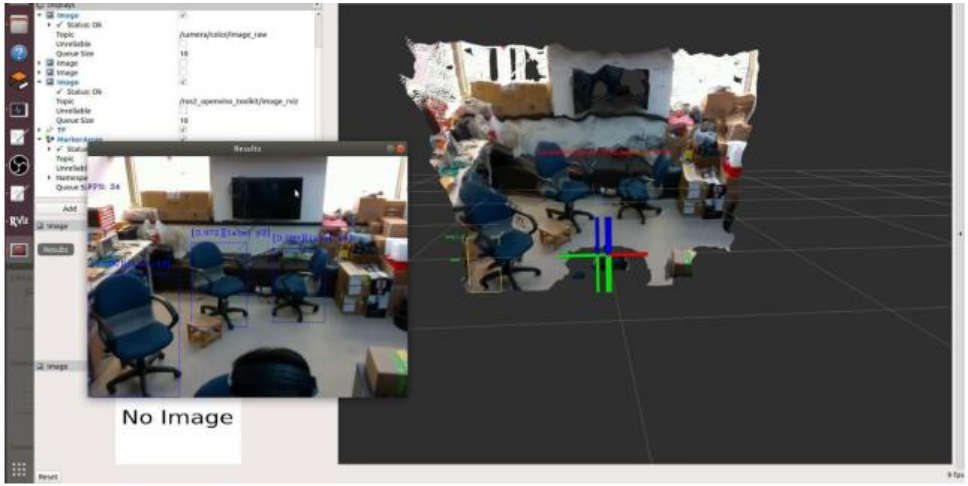
- Liste les nœuds découverts dans le domaine ROS actif.

RVIZ 2

Outil de visualisation

rviz2

- **Contient des visualisations préparées**
  - Modèle 3D d'un robot
  - Repères géométriques
  - Nuages de points
  - Images
  - ....
- **Possibilité d'éditer complètement le layout**
- **Possibilité d'intégrer dans une IHM**

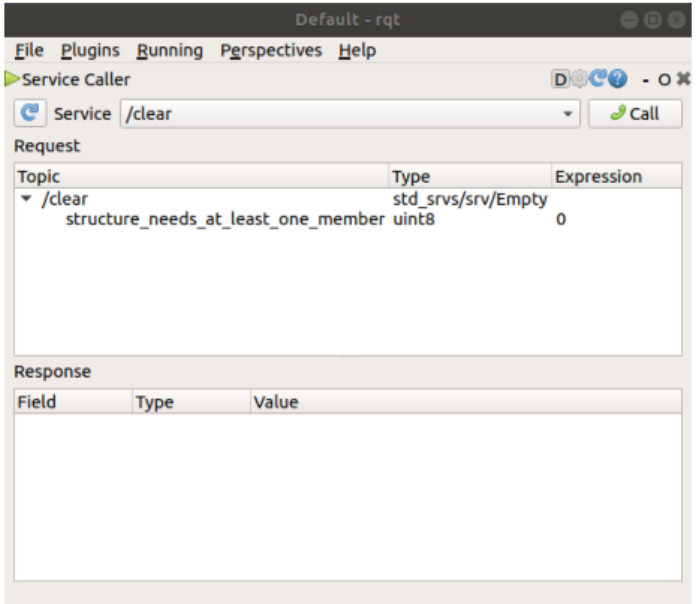


RQT

https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-RQt.html

rqt

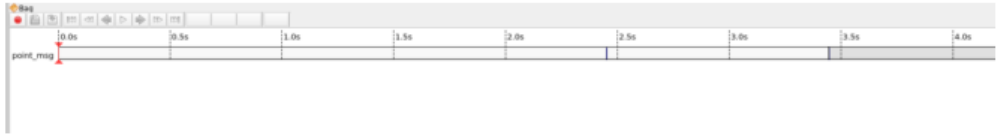
- **Contient beaucoup d'éléments utiles**
  - Loggers
  - Node Graph
  - Publisher
  - Service Caller
  - ...



rosbag2 : enregistrer et rejouer

Enregistrement des topics

ros2 bag



Enregistre les messages et leurs horodatages pour les rejouer.

Utile pour analyser et tester. La relecture ne restaure pas tout l'état du système.

Launch : démarrer un système

Un fichier décrit plusieurs démarrages et leur configuration

Nœuds, paramètres, namespaces et remappings

ros2 launch <package> <fichier_launch>

Centraliser les réglages rend les essais répétables.

Démonstration : observer avant de corriger

Lancer turtlesim et téléopération

Lister les nœuds et les topics ; afficher le graphe

Inspecter le type et écouter un message

Enregistrer un mouvement court puis inspecter le bag.

La méthode de diagnostic

1. Vérifier l’environnement et les nœuds attendus

2. Vérifier noms, types et sens des échanges

3. Vérifier QoS, domaine et circulation des messages

4. Isoler un changement et observer à nouveau.

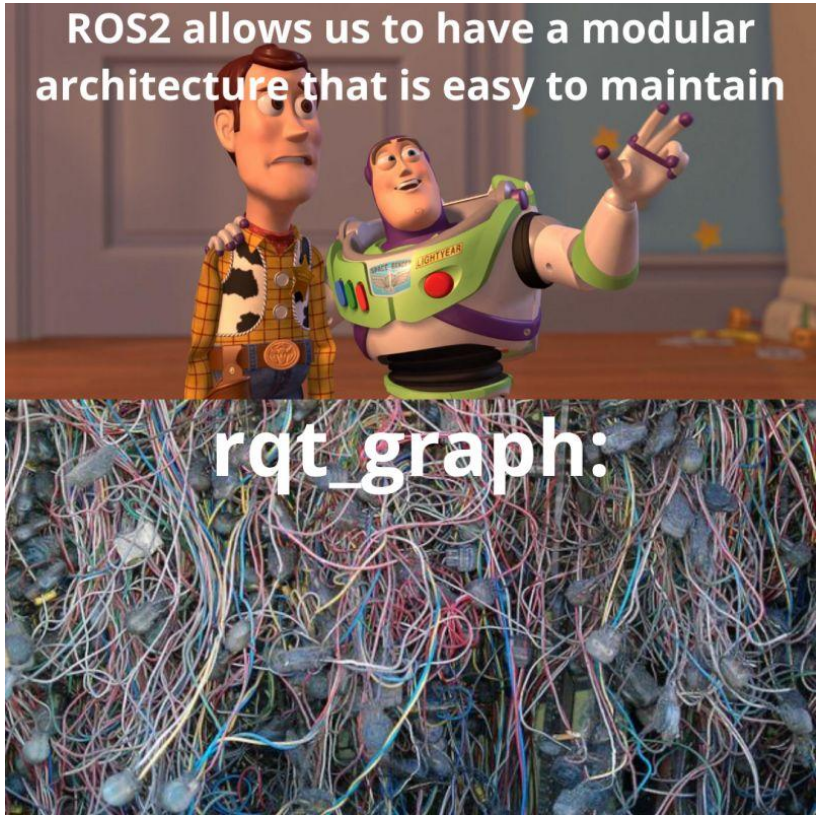
Validation orale des bases

Dessiner deux nœuds reliés par un topic typé

Distinguer package, exécutable et nœud

Donner un exemple de service et d’action

Citer deux causes possibles d’un topic silencieux.


ROS2 allows us to have a modular architecture that is easy to maintain
rqt_graph:

Résumé de la journée

Architecture :
Une responsabilité claire par module.
Des interfaces avec un sens, des délais et des erreurs définis.
Respecter des grands principes de séparations en prenant en compte les coûts

ROS 2 :
Les grandes notions (packages, nœuds, …)
Les échanges ROS 2 choisis selon le besoin.
Quelques outils.